

AngularJS 商品管理系統

1. 簡介

這份教學將指導你如何使用 AngularJS 建立一個簡單的商品管理系統，透過 \$http 服務與 <https://fakestoreapi.com/products> API 互動，實現商品的 CRUD（新增、讀取、更新、刪除）操作。

2. 主要功能

- 讀取商品列表（Read）
- 新增商品（Create）
- 更新商品（Update）
- 刪除商品（Delete）
- 編輯商品資訊

3. 應用程式架構

本專案主要由以下檔案組成：

- `index.html`：負責頁面的結構與 UI
 - `js/main.js`：定義 AngularJS 控制器與 CRUD 操作
-

4. `index.html` 詳細解析

4.1 引入 AngularJS

```
<script  
src="http://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min  
.js"></script>
```

這行程式碼載入 AngularJS 1.8.2 版本，使我們能夠使用 AngularJS 的功能。

4.2 引入 `main.js`

```
<script src="./js/main.js"></script>
```

這行程式碼載入 JavaScript 檔案 main.js，該檔案包含 AngularJS 控制器及 API 請求邏輯。

4.3 定義 AngularJS 應用程式

```
<body ng-app="ProductManagement"  
ng-controller="ProductManagementController">
```

這段程式碼：

- **ng-app="ProductManagement"**：指定 AngularJS 應用程式的模組名稱為 ProductManagement。
- **ng-controller="ProductManagementController"**：指定此頁面使用 ProductManagementController 來管理資料與邏輯。

4.4 商品列表顯示 (ng-repeat)

```
<tr ng-repeat="product in products">  
  <td>{{ product.title }}</td>  
  <td>{{ product.price }}</td>  
  <td></td>  
  <td><a ng-click="editProduct(product)" class="button">Edit</a> | <a  
ng-click="removeProduct(product)" class="button">Remove</a></td>  
</tr>
```

這段程式碼：

- **ng-repeat="product in products"**：遍歷 products 陣列，並將每個商品顯示在表格中。
- **{{ product.title }}**：顯示商品名稱。
- **{{ product.price }}**：顯示商品價格。
- ****：顯示商品圖片。
- **ng-click="editProduct(product)"**：點擊「Edit」按鈕後，商品資訊會填入表單，讓使用者進行編輯。
- **ng-click="removeProduct(product)"**：點擊「Remove」按鈕後，該商品會被刪除。

4.5 新增/編輯商品表單

```

<form ng-submit="submitProduct()">
  <table>
    <tr>
      <td>Category</td>
      <td><input type="text" ng-model="form.category" size="60" /></td>
    </tr>
    <tr>
      <td>Title</td>
      <td><input type="text" ng-model="form.title" size="100" /></td>
    </tr>
    <tr>
      <td>Description</td>
      <td><input type="text" ng-model="form.description" size="100"
/></td>
    </tr>
    <tr>
      <td>Price</td>
      <td><input type="text" ng-model="form.price" size="60" /></td>
    </tr>
    <tr>
      <td>Image Path</td>
      <td><input type="text" ng-model="form.image" size="60" /></td>
    </tr>
    <tr>
      <td colspan="2"><input type="submit" value="Submit" /></td>
    </tr>
  </table>
</form>

```

這段程式碼：

- **ng-submit="submitProduct()"**：當按下提交按鈕時，觸發 submitProduct() 方法。
- **ng-model 綁定表單數據**：例如 ng-model="form.title" 會將輸入框的值存入 form.title。

5. 總結

這份教學手冊詳細解析了 AngularJS 商品管理系統的核心概念，包括 HTML 佈局、AngularJS 控制器與 API 請求。透過這份指南，你可以學會如何使用 AngularJS 與 API 進行資料操作，並將其應用於更複雜的專案中。

希望這份教學手冊對你有所幫助！

main.js

1. 簡介

這段程式碼使用 AngularJS 來實現商品管理系統，並與 <https://fakestoreapi.com/products> 進行互動，提供 CRUD（Create、Read、Update、Delete）功能。

2. 主要功能

- 讀取商品列表（Read）
- 新增商品（Create）
- 更新商品（Update）
- 刪除商品（Delete）
- 編輯商品資訊

3. AngularJS 基本設定

在 AngularJS 中，`var app = angular.module("ProductManagement", [ ])`；這段程式碼用來建立一個名為 **ProductManagement** 的模組，而 `[ ]`（空陣列）的作用是 定義該模組的相依性（dependencies）。

`[ ]` 的用途

- 指定該模組所依賴的其他 AngularJS 模組
當一個 AngularJS 應用程式需要使用額外的功能（例如 `ngRoute` 用於路由，`ngAnimate` 用於動畫等），可以在這個陣列中列出這些模組。例如：

- `var app = angular.module("ProductManagement", ["ngRoute", "ngAnimate"]);`

這樣 `ProductManagement` 模組就會依賴 `ngRoute` 和 `ngAnimate`，在應用程式中可以使用這些模組的功能。

- 若為空陣列 `[]`，表示該模組沒有額外的依賴模組
當 `[]` 為空時，代表這個模組是獨立的，不依賴其他 AngularJS 模組。這樣的寫法適用於簡單的應用程式，或者是後續會再加入依賴的模組。

舉例

1. 無相依性模組
2. `var app = angular.module("MyApp", []);`
 - `MyApp` 是一個獨立的 AngularJS 模組，沒有任何額外的依賴。
3. 帶有相依性的模組
4. `var app = angular.module("MyApp", ["ngRoute", "ngCookies"]);`
 - `MyApp` 需要 `ngRoute`（處理路由功能）與 `ngCookies`（處理 Cookie 存取）。

總結

`[]` 的作用是**指定該模組所依賴的其他 AngularJS 模組**，如果沒有相依性，就填入空陣列 `[]`，代表該模組是獨立的。這種方式讓 AngularJS 應用程式的架構更加模組化，方便日後擴展與維護。

4. 控制器 `fetchProducts`

這個控制器 `fetchProducts` 是 `ProductManagement` 應用程式的核心部分，負責處理商品的 CRUD 操作。

```
app.controller("ProductManagementController", fetchProducts);
```

這行程式碼將 `fetchProducts` 控制器註冊到 AngularJS 應用程式中。

4.1 變數初始化

```
$scope.products = [];
```

```
$scope.form = {  
  id : -1,  
  category : "",  
  description : "",  
  price : "",  
  images:"",  
  title:"",  
  rating:{}  
};
```

這裡定義了 `products` 陣列來存放商品數據，並初始化 `form` 物件來儲存商品表單資訊。

4.2 讀取商品列表

```
function _refreshPageData() {  
  $http({  
    method : 'GET',  
    url : 'https://fakestoreapi.com/products'  
  }).then(function successCallback(response) {  
    $scope.products = response.data;  
  }, function errorCallback(response) {  
    console.log(response.statusText);  
  });  
}
```

這個函式 `_refreshPageData()` 會向 API 發送 GET 請求，取得所有商品資訊並存入 `$scope.products` 陣列。

4.3 新增或更新商品

```
$scope.submitProduct = function() {  
  var method = "";  
  var url = "";  
  if ($scope.form.id == -1) {  
    method = "POST";  
    url = 'https://fakestoreapi.com/products';  
  } else {  
    method = "PUT";  
    url = 'https://fakestoreapi.com/products/' + $scope.form.id;
```

```

    }

    $http({
      method : method,
      url : url,
      data : angular.toJson($scope.form),
      headers : {
        'Content-Type' : 'application/json'
      }
    }).then( _success, _error );
  };

```

這個函式 submitProduct() 會根據 id 來決定是新增 (POST) 還是更新 (PUT) 商品。

- **新增商品**：若 id == -1，則執行 POST 請求。
- **更新商品**：若 id 存在，則執行 PUT 請求。

4.4 刪除商品

```

$scope.removeProduct = function(product) {
  $http({
    method : 'DELETE',
    url : 'https://fakestoreapi.com/products/' + product.id
  }).then(_success, _error);
};

```

這個函式 removeProduct() 根據 id 執行 DELETE 請求來刪除商品。

4.5 編輯商品資訊

```

$scope.editProduct = function(product) {
  $scope.form.category = product.category;
  $scope.form.title = product.title;
  $scope.form.description = product.description;
  $scope.form.price = product.price;
  $scope.form.rating = product.rating;
  $scope.form.image = product.image;
  $scope.form.id = product.id;
};

```

這個函式 `editProduct()` 允許使用者點擊某個商品時，將商品資訊填入 form，準備進行編輯。

4.6 請求成功與錯誤處理

```
function _success(response) {  
    _refreshPageData();  
    _clearForm();  
}  
  
function _error(response) {  
    console.log(response.statusText);  
}
```

- `_success()`：請求成功時，重新載入商品列表，並清空表單。
- `_error()`：請求失敗時，在控制台顯示錯誤訊息。

4.7 清空表單

```
function _clearForm() {  
    $scope.form.category = "";  
    $scope.form.title = "";  
    $scope.form.description = "";  
    $scope.form.price = "";  
    $scope.form.rating = "";  
    $scope.form.image = "";  
    $scope.form.id = -1;  
};
```

這個函式 `_clearForm()` 會將表單欄位清空，方便使用者輸入新商品資訊。

5. 總結

這份 AngularJS 程式碼實現了一個完整的商品管理系統，具備 CRUD 操作，並透過 FakeStoreAPI 進行資料存取。開發者可以根據這個範例，進一步優化 UI 介面，或連結自己的後端 API 來實現真實的應用程式。

main.js

```
function fetchProducts($scope, $http){
    //Initialize page with default data which is blank in this example
    $scope.products = [];
    $scope.form = {
        id : -1,
        category : "",
        description : "",
        price : "",
        images:"",
        price:"",
        title:"",
        rating:{}
    };

    //Now load the data from server
    _refreshPageData();

    //HTTP POST/PUT methods for add/edit employee
    $scope.submitProduct = function() {
        var method = "";
        var url = "";
        if ($scope.form.id == -1) {
            //Id is absent so add employee - POST operation
            method = "POST";
            url = 'https://fakestoreapi.com/products';
        } else {
            //If Id is present, it's edit operation - PUT operation
            method = "PUT";
            url = 'https://fakestoreapi.com/products/' + $scope.form.id;
        }

        $http({
            method : method,
            url : url,
            data : angular.toJson($scope.form) ,
            headers : {
                'Content-Type' : 'application/json'
            }
        })
    }
}
```

```

    }
  }).then( _success, _error );
};

//HTTP DELETE- delete employee by Id
$scope.removeProduct = function(product) {
  $http({
    method : 'DELETE',
    url : 'https://fakestoreapi.com/products/' + product.id
  }).then(_success, _error);
};

//In case of edit employee, populate form with employee data
$scope.editProduct = function(product) {
  $scope.form.category = product.category;
  $scope.form.title = product.title;
  $scope.form.description = product.description;
  $scope.form.price = product.price;
  $scope.form.rating = product.rating;
  $scope.form.image = product.image;
  $scope.form.id = product.id;
};

/* Private Methods */
//HTTP GET- get all employees collection
function _refreshPageData() {
  $http({
    method : 'GET',
    url : 'https://fakestoreapi.com/products'
  }).then(function successCallback(response) {
    $scope.products = response.data;
  }, function errorCallback(response) {
    console.log(response.statusText);
  });
}

function _success(response) {
  _refreshPageData();
}

```



```

        color: blue;
    }
    td,th{
        border: 1px solid gray;
        width: 25%;
        text-align: left;
    }
    table {
        width: 800px;
    }
</style>
<head>
    <body ng-app="ProductManagement"
ng-controller="ProductManagementController">
    <h1>
        AngularJS - Use $http to invoke Product APIs
    </h1>

    <table>
        <tr>
            <th>Title</th>
            <th>Price</th>
            <th>Image</th>
            <th>Actions</th>
        </tr>

        <tr ng-repeat="product in products">
            <td>{{ product.title }}</td>
            <td>{{ product.price }}</td>
            <td></td>
            <td><a ng-click="editProduct( product )" class="button">Edit</a> | <a
ng-click="removeProduct( product )" class="button">Remove</a></td>
        </tr>

    </table>

    <h2>Add/Edit Product</h2>

```

```
<form ng-submit="submitProduct()">
  <table>
    <tr>
      <td>Category</td>
      <td><input type="text" ng-model="form.category" size="60" /></td>
    </tr>
    <tr>
      <td>Title</td>
      <td><input type="text" ng-model="form.title" size="100" /></td>
    </tr>
    <tr>
      <td>Description</td>
      <td><input type="text" ng-model="form.description" size="100"
/></td>
    </tr>
    <tr>
      <td>price</td>
      <td><input type="text" ng-model="form.price" size="60" /></td>
    </tr>
    <tr>
      <td>Image Path</td>
      <td><input type="text" ng-model="form.image" size="60" /></td>
    </tr>
    <tr>
      <td colspan="2"><input type="submit" value="Submit" /></td>
    </tr>
  </table>
</form>
</body>
</html>
```